# AMMBER - Motivational Modelling Local Editor

Progress Report - Weeks 1-6 (27 Jul - 6 Sep 2026)

Team Deepsick · Client: Leon Sterling · Tutor: Kerr Yuan

| Member | GitHub | Role |
|---|---|---|
| Jinbao Liu | lkevincc0 | Tech Lead / Full-stack Developer (architecture, CI/CD, integration, client liaison) |
| Zihan Xie | photozity-a11y (zihxie1) | Frontend Developer (canvas UI, image import/export, feedback integration) |
| Huimin Peng | ViolaHM-14 | Frontend Developer (feedback panel, node binding, profile/replies) |
| Zecheng Huang | ZechengHuang-unimelb | QA / Test Lead / Developer (tests, bug triage, Jira and documentation records, sharing) |
| Yuhao Xu | yuhaox5 | DevOps & Tooling / Developer (CI/CD, dependencies and deployment tooling) |

## 1 Project Overview

AMMBER is a web-based motivational modelling tool used in teaching and research for roughly fifteen years. We are extending the existing React/TypeScript editor (fork: `github.com/lkevincc0/mm-local-editor`) so that lecturers and reviewers can give structured feedback on motivational models, and so that models can be shared as portable files. In Week 3, the client confirmed that neither a cloud service nor a shared database was required. The team therefore removed the backend from scope and adopted local browser storage.

Delivered in Weeks 1–6 (implementation evidence below; the develop branch is continuously deployed to GitHub Pages):

- **Local project workspace** - create/open/rename/delete projects persisted in-browser, no server (PR #3).
- **Unified editorial theme & dockable canvas editor** - consistent visual identity; canvas fits and centres the model on load (PRs #3, #4).
- **Feedback feature end-to-end** - closable draw.io-style side panel, node-bound comments with click-to-locate, authoring, nested replies, editable profile header, generated avatars (PRs #5, #6, #9, #11, #19).
- **Image-only project import/export** - project JSON (name + feedback) embedded invisibly into exported PNG/SVG, so one image is a full portable backup (PR #7); compact share links via deflate compression (PR #20).
- **QR-code project sharing** - modernised share dialog inside the editor (PR #15).
- **Deployment & robustness** - adaptive router basename, GitHub Pages asset paths, deploy permissions (PRs #2, #12, #13).
- **Quality pipeline** – 20 Vitest unit/component test files and one Cypress E2E specification, all run by GitHub Actions; four reported issues fixed with regression coverage (PRs #14, #17, #18, #23).

## 2 Process

### 2.1 Agile Ceremonies

After an initial Sprint 0 for project setup, the team worked in two-week sprints. Dated sprint plans, weekly minutes, retrospectives and decisions are published in the Confluence evidence index; Jira is the executable record, where stories, tasks and bugs carry owners, acceptance criteria and implementation evidence.

- Weekly team meetings during Weeks 3–6, with attendance, agenda, outcomes and owned action items recorded in the minutes: W3, W4, W5 and W6.
- Direct ceremony records: Sprint 1 planning, Sprint 1 retrospective, Sprint 2 planning and Sprint 2 retrospective. The Sprint 1 actions on PR size, role clarity and fresh-clone verification were adopted during Sprint 2.
- Client consultation at least fortnightly; requests, resulting Jira work and implementation status are recorded in the client-feedback log (Section 2.3).
- Sprints: S0 (W1–2) kick-off – completed; S1 (W3–4) Git workflow and first vertical slice – completed; S2 (W5–6) feedback, image portability and sharing – current, with the overall-feedback/export change still under review in PR #27.

### 2.2 Decision Making, Meetings and Internal Communication

Major decisions are discussed by the team, with options, rationale and outcome recorded in the decision log. Seven decisions fall within Weeks 1–6. Their implemented outcomes are independently visible in Jira and GitHub: the local-only architecture (KAN-18), closable feedback panel (KAN-12), node-bound feedback (KAN-13), and portable image data (KAN-15).

Communication uses Slack for daily coordination, GitHub PR reviews for technical discussion, and Jira for exe-

cutable work records. Git history provides concrete co-authored collaboration evidence: PR #7 combines Zihan's image-metadata work with Jinbao's integration commit; the PR #13 deployment fix credits Jinbao, Zecheng and Yuhao; PR #17 combines Yuhao's import fix with Jinbao's regression test; and PR #23 combines Zecheng's implementation/tests with Yuhao co-credit. These records support co-authored collaboration and cross-review, not a claim of continuous pair programming.

### 2.3 Communication with the Client

| Week | Client feedback | Impact | Status |
|---|---|---|---|
| W3 | No cloud service or shared database required | Backend removed from scope; local browser storage prioritised; QA/testing and DevOps/tooling responsibilities expanded | Actioned (S1) |
| W4 | Prioritise feedback workflow over extra export formats | Feedback moved to top of S2 backlog | Actioned (S2) |
| W5 | Persistence, identity, sharing and feedback export | Jira tasks; PRs #17/#19/#20/#23 merged; #27 open | Partly actioned |
| W6 | S2 demo accepted; packaged deployment and user documentation requested | Deployment and documentation work recorded in the backlog | Recorded |

## 3 Artefacts

### 3.1 Requirements

Requirements are recorded as Jira user stories with acceptance criteria and implementation evidence. The reviewer workflow is traceable from requirement to code: a closable canvas feedback panel (KAN-12 → PR #5 and PR #6), node-bound comments with click-to-locate (KAN-13 → PR #6), and authoring, replies and profiles (KAN-14 → PR #9 and PR #11). Portable feedback is specified in KAN-15–KAN-16 and implemented in PR #7; the feedback-restoration bug is tracked in KAN-21 and fixed with regression coverage in PR #17. The local-only constraint is recorded in KAN-18.

### 3.2 Frontend and Architectural Design

The interface uses a warm, minimal editorial theme (warm paper `#f7f6f0`, deep purple `#25213e` and accent `#6b51c9`). The feedback interface is a closable side panel so reviewers remain on the canvas; selecting a comment locates its model element. The dockable canvas workspace was introduced in PR #3, the feedback panel and node binding in PRs #5 and #6, and image-based project portability in PR #7.

The application follows the local-first decision recorded in the decision log and KAN-18. It is a React 18/TypeScript/Vite single-page application: maxGraph provides the canvas, Redux Toolkit manages model state, and the project provider coordinates project data and feedback persistence. React Router uses an adaptive basename so the same build works locally and under the GitHub Pages sub-path. Project data, feedback and profile settings remain in browser storage, while image metadata utilities embed portable project data in PNG/SVG files. No backend or shared database is required.

### 3.3 Coding, Testing, Deployment and Repository

- **Coding:** Repository conventions introduced in PR #1 document TypeScript/React naming, tests, branch names and PR expectations (`AGENTS.md`). ESLint is configured with a zero-warning policy. After the workflow was established, feature and bug-fix changes were submitted through pull requests and reviewed before merging (merged PR record).
- **Testing:** The repository contains 20 Vitest unit/component test files co-located with source code and one Cypress E2E specification; both suites run in GitHub Actions. Regression tests cover project rename (#14), feedback import/persistence (#17), Do3 task handling (#18), and node redraw/feedback preservation (#23). Zecheng maintains the QA test plan and Jira bug records as coverage changes.
- **Deployment:** Pushes to develop run the CI workflow; successful builds are published by the CD job to GitHub Pages. The deployment workflow is visible in `ci-cd.yml`, and routing/asset fixes are evidenced by PRs #2, #12 and #13. The project can also be launched locally via Docker Compose.
- **Repository:** A GitFlow-lite model is documented in `AGENTS.md`. Twenty pull requests had been merged during the reporting period as of 4 Sep. Repository maintenance included removing an unintended submodule entry (PR #8), updating lockfiles and retaining an environment template.

## 4 Maintenance of Artefacts and Response to Feedback

The dated sprint plans, weekly minutes, retrospectives and decisions were maintained in the team agile workbook and are now published and indexed in the Confluence evidence index. Jira maintains the corresponding require-

ments, tasks and bugs; Zecheng maintains the Jira and documentation records while issue owners update their assigned work. The Week 3 local-only change is traceable to KAN-18, and feedback requirements are linked to implementation evidence (feedback/local requirement set). The team checks Jira items against the commit history and merged PR record; bug tickets link to the corresponding fixes.

The 26 Aug feedback review was translated into specific Jira work rather than left as narrative notes: feedback restoration (KAN-21 → PR #17), the assigned UI/status correction (KAN-23 → PR #18), editable commenter identity (KAN-34 → PR #19), compact sharing (KAN-35 → PR #20), and redraw preservation (KAN-37 → PR #23). The remaining overall-feedback/export request is tracked in KAN-33 and implemented in PR #27, which is still open and is not counted as delivered. The complete status chain is summarised in the client-feedback log.

Special circumstances: the Week 3 local-only clarification removed backend work and shifted responsibilities toward QA/testing and DevOps/tooling; an unintended gitlink that prevented reliable fresh clones was removed in PR #8; Sprint 2 follow-up work was distributed across all five members, with the overall-feedback/export change still under review in PR #27.